



Operating Systems



Course Code:BIT253CO
Year/ Semester: II/IV

Compiled by Shishir Ghimire

Credit Hours: 3hrs



Unit - 01: Introduction

Class Load : 3 hrs

► TABLE OF CONTENTS

1. Introduction

- a. Operating system as an extended machine & resource manager
- b. History and types of operating system
- c. Operating system concepts, functions, structures

[3 Hrs]

► Introduction:



► Computer System:

❖ Computer System can be divided into **four components**:

1. **Hardware:**

- Provides basic computing resources: CPU, Memory, I/O Devices

2. **Operating System**

- Controls and coordinates use of hardware among various applications and users

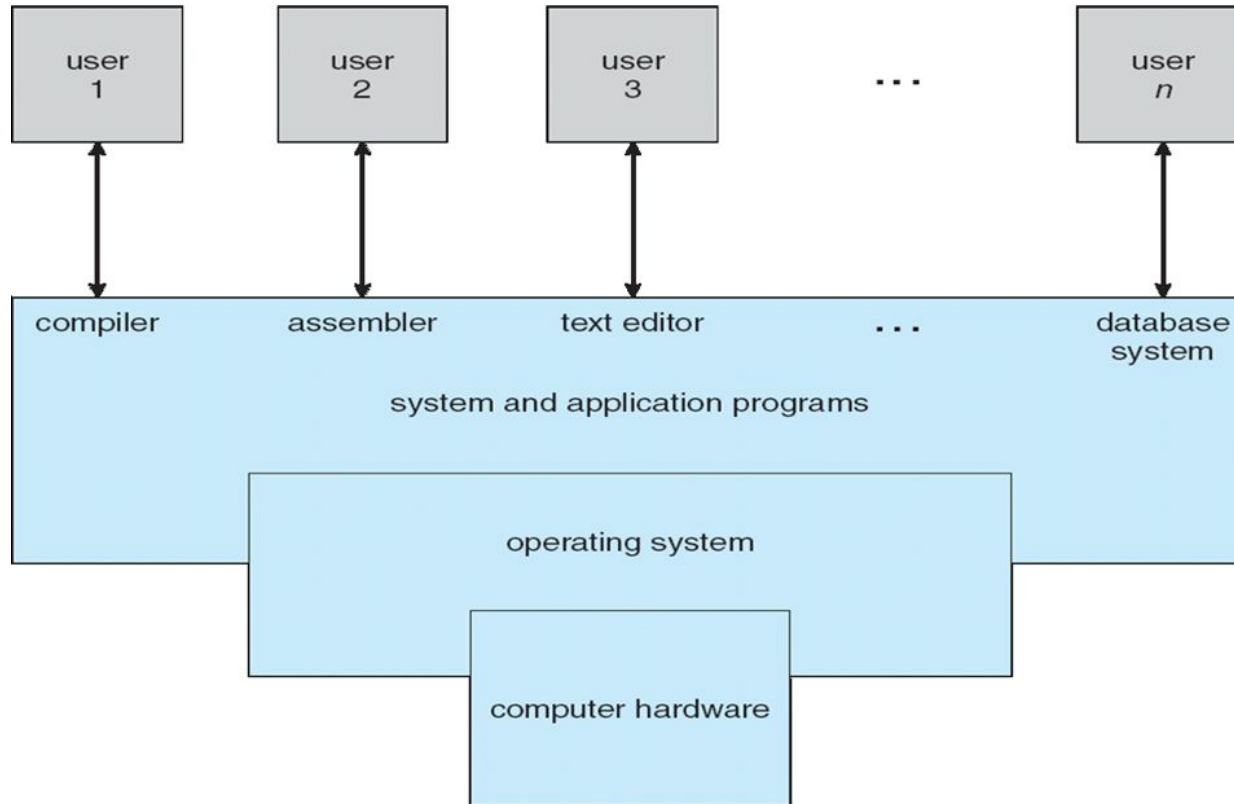
3. **Application Programs:**

- Define the ways in which the system resources are used to solve the computing problems of the users
- Word processors, compilers, web browsers, database systems, video games

4. **Users**

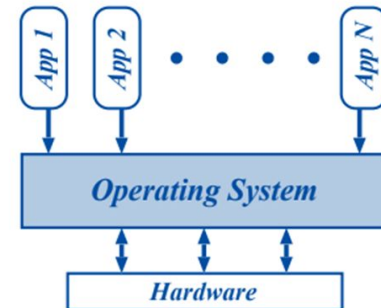
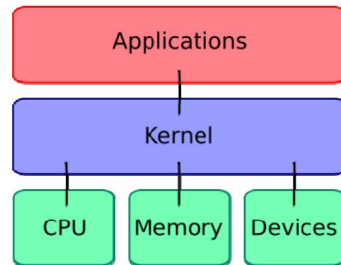
- People, machines, other computers

► Computer System:



► Operating System (OS):

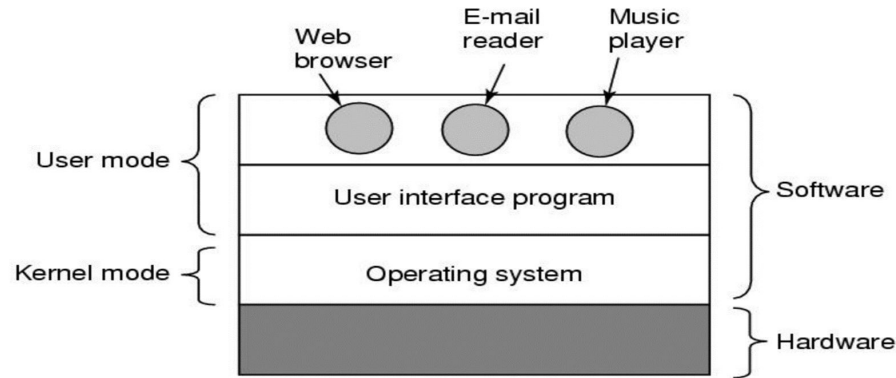
- ❖ An operating system (OS) is a collection of **system programs** that together control the operation of a computer system.
- ❖ It is the **infrastructure software component of a computer system** responsible for the **management and coordination of activities and the sharing** of the resources of the computer.
- ❖ OS offers a simplified **man-machine interface (MMI)** to overcome the complexity of actual machine.
- ❖ The core of the operating system is the **kernel**, a control program that functions in privileged state (an execution context that allows all hardware instructions to be executed), reacting to interrupts from external devices and to service requests and traps from processes.



► Operating System (OS):

◆ Two modes of operation:

- **Kernel Mode:** Kernel Mode is the privileged mode where the **core part of the operating system, the kernel** executes. It has unrestricted access to all machine resources including CPU, memory, storage, and connected device
- **User Mode:** Application software runs in user mode. Programs in **user mode can't access hardware or kernel memory directly**. They must request access through system calls to the kernel.



► Operating Systems► Layers and Views:

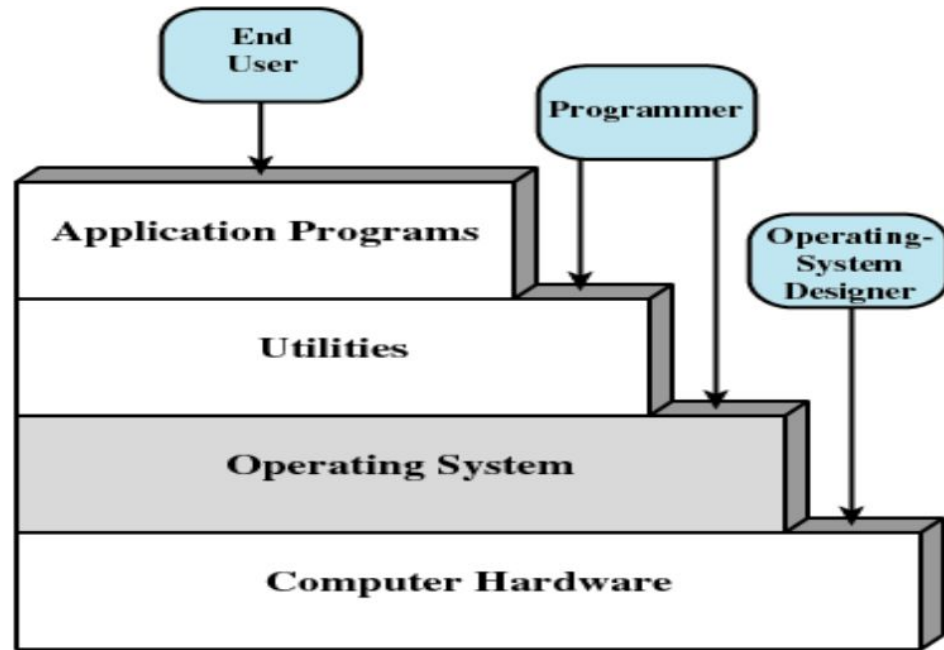


Figure 2.1 Layers and Views of a Computer System

► Operating Systems ► **Two View** of Operating System:

❖ The two main views of an operating system are

► User View

- How regular users see and interact with the OS?
- The OS provides an **easy-to-use interface**, like Windows (GUI) or Unix (CLI or Terminal).
- Different devices (phones, computers, servers) have different OS designs for ease or performance.

► System View

- How the **OS works behind the scenes** to manage the computer?
 - OS is a Resource Allocator.
 - OS is Control Program.

► Operating Systems:

◆ Operating System Goals

- Implementing the user interface
- Sharing hardware among users
- Allowing users to share data among themselves
- Preventing users from interfering with one another
- Scheduling resources among users
- Facilitating input/output
- Recovering from errors
- Accounting for resource usage
- Facilitating parallel operations
- Organizing data for secure and rapid access
- Handling network communications

► Operating Systems ► **Two function of OS :**

- ❖ The two major functions of OS are:
 1. OS as an Extended Machine
 2. OS as a Resource Manager

► Operating Systems ► Two function of OS:

◆ OS as an Extended Machine :

- OS creates a high level of abstraction for the programmer.
 - **Example:** (Floppy Disk I/O Operation)
 - Disk contains a list of named files.
 - Each file can be opened for read write.
 - After read write is complete, close that file.
 - No any detail to deal.
- The abstraction **hides the truth about the hardware from the programmer** and presents a **simple view of the named files** that can be read and written.
- OS prevents the user with the equivalent of an extended machine or virtual machine that is easier to program than the underlying hardware.

► OS as an Extended Machine :

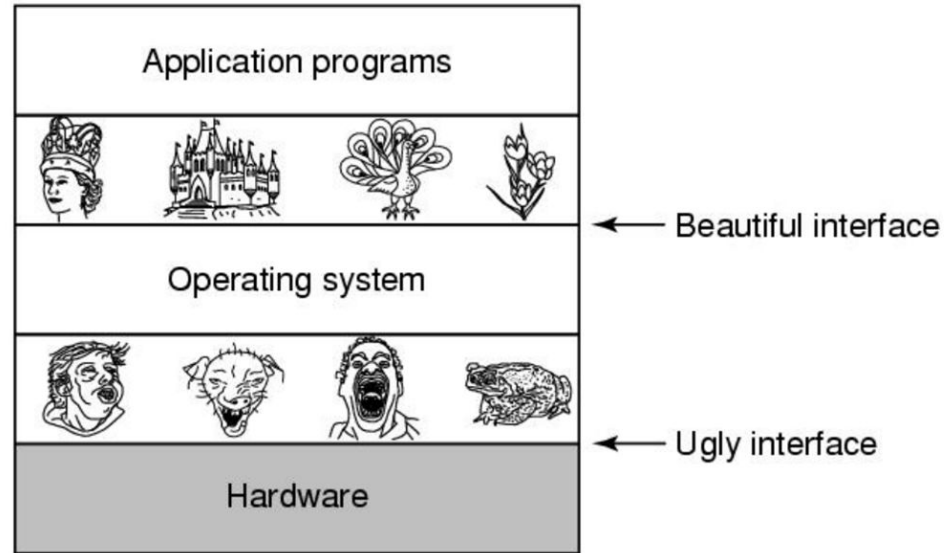


Figure 1-2. Operating systems turn ugly hardware into beautiful abstractions.

► Operating Systems ► Two function of OS:

◆ OS as a Resource Manager:

- What happens if three programs try to print their output on the same printer at the same time?
- What happens if two network users try to update a shared document at same time?
 - OS primary function is to keep track of who is using which resource to grant resource requests, to account for usage and to mediate conflicting requests from different programs and users.
- Virtualizes resources so multiple users can share it.
- Protect application from one another.
- Provide efficient and fair access to resource.

► OS as a Resource Manager

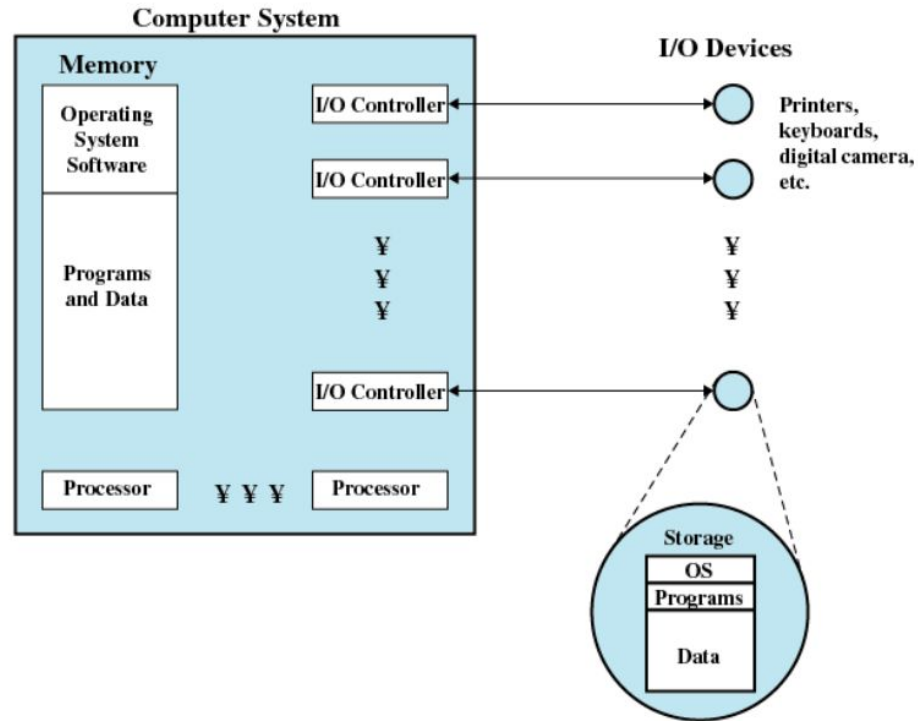


Figure 2.2 The Operating System as Resource Manager

► Evolution/History of Operating System:

- ❖ Present OS haven't been developed overnight.
- ❖ Operating systems have evolved through a number of distinct phases or generations which corresponds roughly to the decades.
- ❖ The evolution of OS can be summarized in table below as:

Generation of computer	Time Period	Major components	Types of Operating Systems (OS)
First Generation	1945-1955	Vaccum Tube	No OS (Manual Operation)
Second Generation	1956-1964	Transistor	Batch Operating System
Third Generation	1965-1971	Integrated Circuit	Multiprogramming Operating System
Fourth Generation	1972-Present	Microprocessor	Time-Sharing Operating System, Real-Time OS, Network OS
Fifth Generation	Future	Bio-Chips	AI-based OS, Natural Language OS, Ubiquitous OS

► Evolution/Generation of Operating System:

❖ First Generation (1945-55), (OS : User Driven)

- **Vacuum-tubes** as building machine.
- Single group of people designed, built, programmed, operated and maintained each machine.
- **Machine Code** is used.
- OS and Programming languages **were unknown (not even assembly languages)**.
- Human operators **scheduled** jobs for execution and **supervise** the use of the computer's resources.
- Machines of the time were so primitive that programs were often entered one bit at time on rows of mechanical switches (plug boards).
- The earliest electronic digital computers had no operating systems.

► Evolution/Generation of Operating System:

◆ Second Generation (1955-64) , (OS : Batch Processing)

- **Transistors** as building machine
- Operating System: **Batch Processing OS**
 - Batch is defined as a **group of jobs with similar needs**. (Example : payroll system where salary calculations for all employees are collected together and processed)
 - OS allows user to **form batches** .
 - Computer execute **each batch sequentially** , processing all jobs of a batch as a **single process called batch processing**.
 - Largely programmed in **FORTRAN** and **assembly language**.
 - **Example:** FORTRAN card, telling the operating system to load the FORTRAN compiler from the system tape.

► Second Generation:

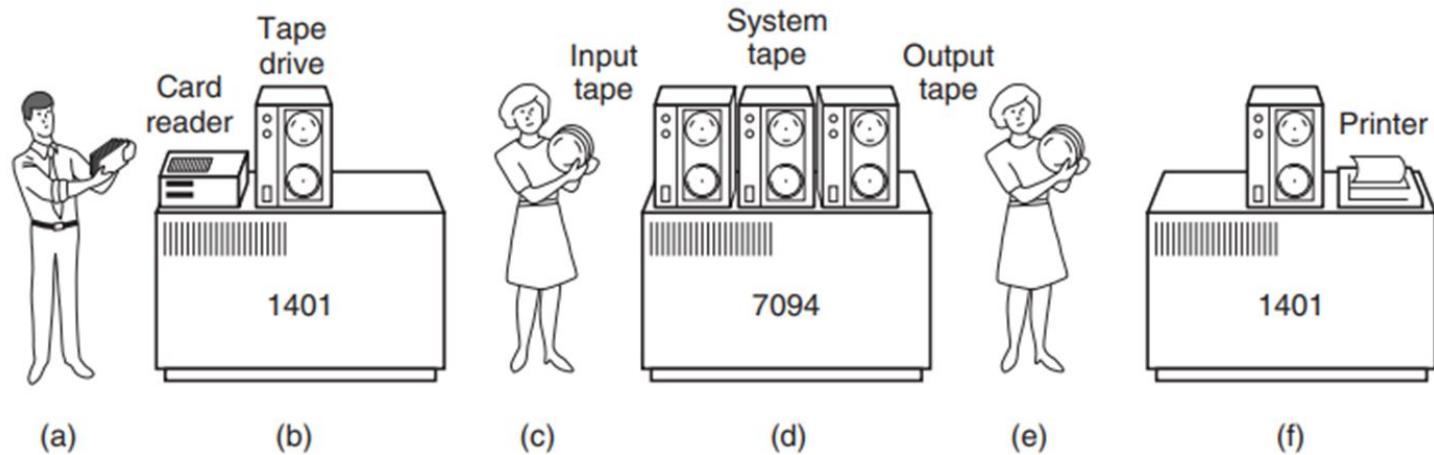


Figure 1-3. An early batch system. (a) Programmers bring cards to 1401. (b) 1401 reads batch of jobs onto tape. (c) Operator carries input tape to 7094. (d) 7094 does computing. (e) Operator carries output tape to 1401. (f) 1401 prints output.

► Evolution/Generation of Operating System:

❖ Third Generation (1965-71) (OS : Multiprogramming)

- **Integrated Circuit (IC)** as building machine
- **Multiprogramming** came into existence
- Operating System: **Multiprogramming OS**
 - Operating systems that are able to **execute more than one program simultaneously** on a processor.
 - A **single program cannot keep CPU or I/O devices busy** at all times.
 - Increases CPU utilization by organizing jobs .
 - **Example:** One program is waiting for an I/O transfer, another program can use the processor.

► Evolution/Generation of Operating System:

◆ The Fourth Generation (19772 to present), (Time Sharing OS, Real Time OS, Network OS)

- **LSI and VLSI** as building machine
- With the development of LSI (Large Scale Integration) circuits, chips, operating system entered in the system entered in the **personal computer and the workstation age**.
- Microprocessor Technology evolved to the point that it becomes **possible to build** desktop computers as **powerful** as the mainframes of the 1970s
- **Operating Systems: IBM PC (MS-DOS), Apple Macintosh (Macintosh)**

► Types of Operating System:

- ❖ The types of Operating System are listed below:
 - Batch Operating System
 - Multiprogramming Operating System
 - Time Sharing Operating System
 - Multiprocessing Operating System
 - Real-Time Operating System
 - PC Operating System
 - Server Operating System
 - Mainframe Operating System
 - Distributed Operating System
 - Mobile Operating System
 - Network Operating System and so on.

► Types of Operating System ► Batch Operating System :

- ❖ Batch processing is a technique in which **operating system collects one programs and data together** in a batch before processing starts.
- ❖ Operating system does the following activities related to batch processing.
- ❖ OS defines a job which has predefined sequence of commands, programs and data as a single unit.
- ❖ OS keeps a number a **jobs in memory and executes** them without any manual information.
- ❖ Jobs are processed in the order of submission i.e **first come first served** fashion.
- ❖ When job completes its execution, its memory is released and the output for the job gets copied into an outputs pool for later printing or processing.

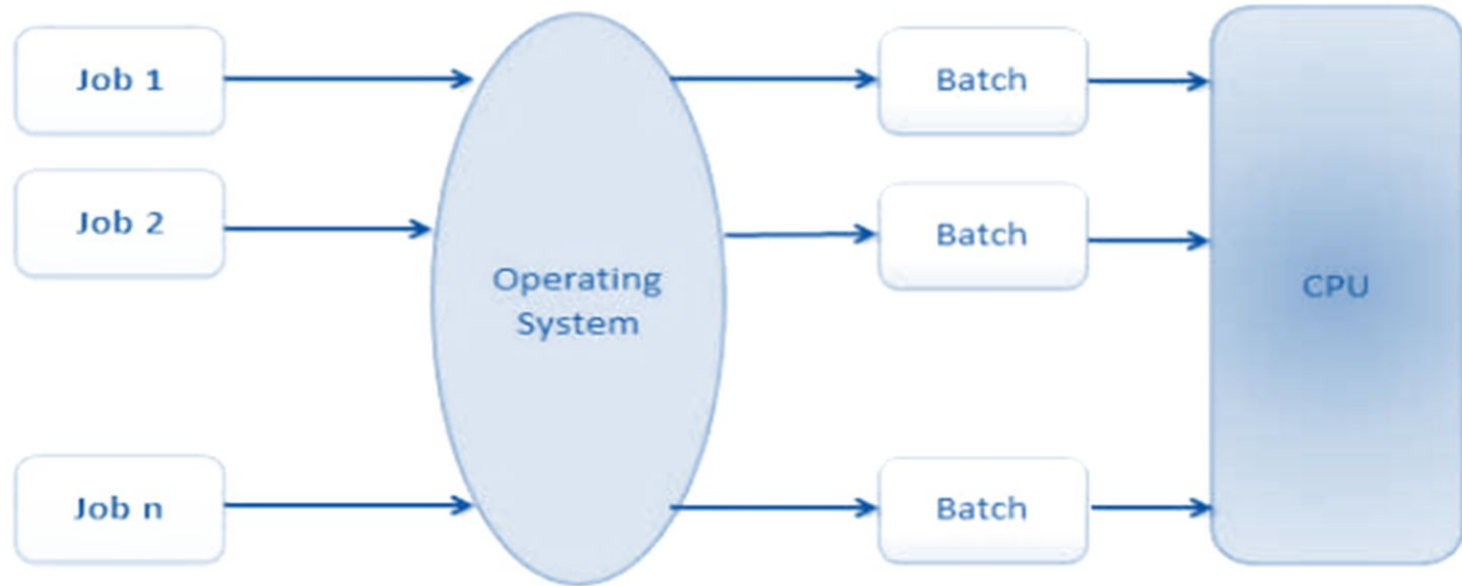
❖ ADVANTAGES

- Batch processing takes much of the work of the operator to the computer.
- Increased performance as a new job get started as soon as the previous job finished without any manual intervention.

❖ DISADVANTAGES

- Difficult to debug program.
- A job could enter an infinite loop.
- Due to lack of protection scheme, one batch job can affect pending jobs.

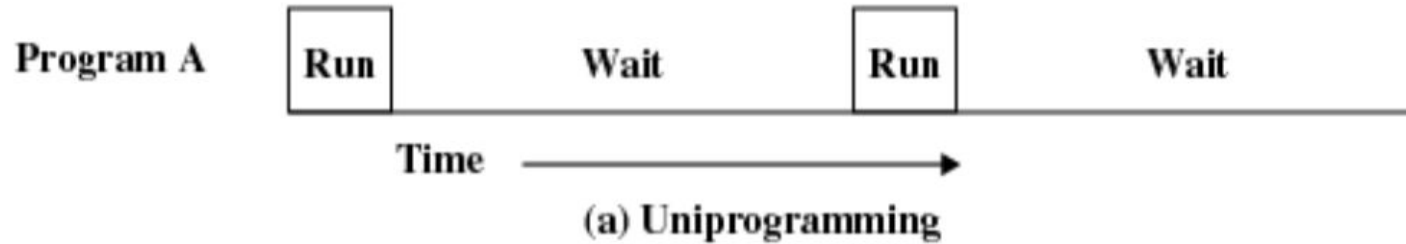
► Types of Operating System ► **Batch Operating System:**



► Types of Operating System > **Multiprogramming Operating System** :

❖ Uniprogramming

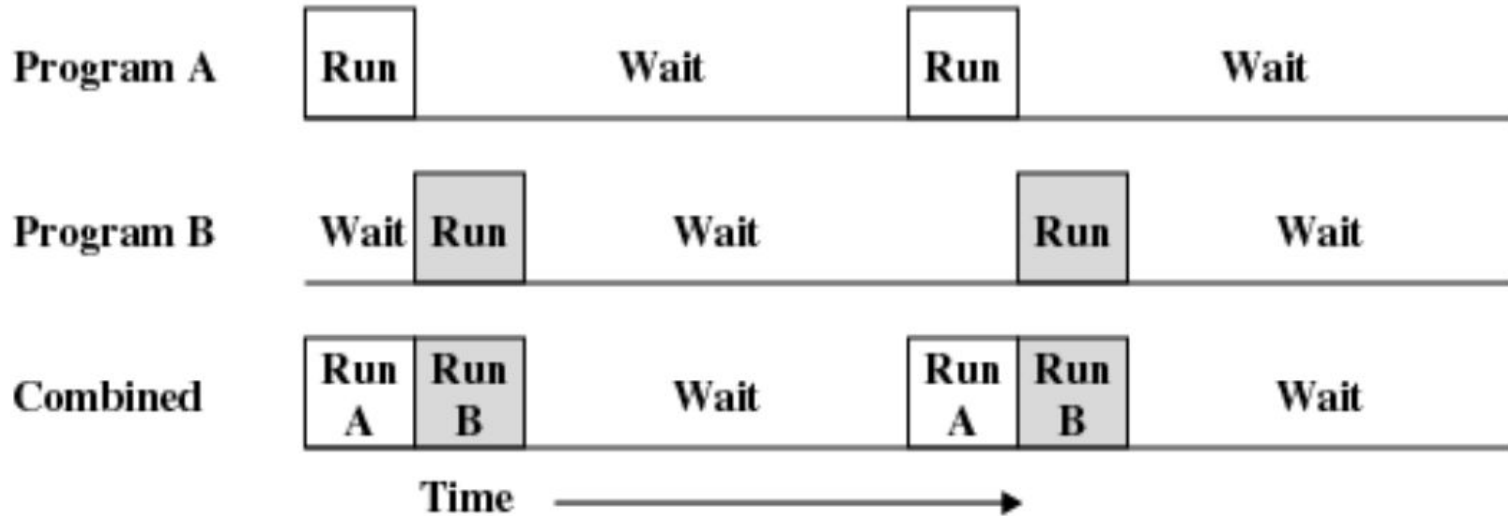
- Processor must wait for I/O instruction to complete before proceeding.



► Types of Operating System ► Multiprogramming Operating System:

❖ Multiprogramming

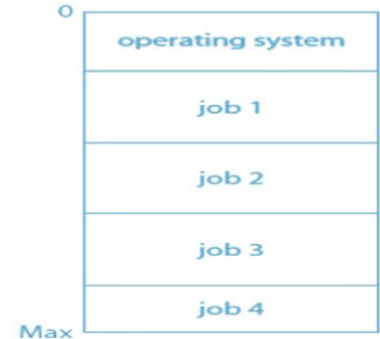
- When one job needs to wait for I/O, the processor can switch to the other job



(b) Multiprogramming with two programs

► Types of Operating System ► Multiprogramming Operating System :

- ❖ When two or more programs are **residing in memory at the same time**, then sharing the processor is referred to the multiprogramming.
- ❖ Multiprogramming assumes a single shared processor.
- ❖ Multiprogramming increases CPU utilization by **organizing jobs** so that the CPU **always has one to execute**.
- ❖ Following figure shows the memory layout for a multiprogramming system.
- ❖ Operating system does the following activities related to multiprogramming.
 - The operating system keeps several jobs in memory at a time.
 - Set of jobs is a subset of the jobs kept in the job pool.
 - The operating system **picks and begins to execute one of the job** in the memory.
 - Multiprogramming operating system monitors the state of all active programs and system resources using memory management programs to ensure that the **CPU is never idle unless** there are no jobs.



► Types of Operating System ► **Multiprogramming Operating System** :

❖ **ADVANTAGES**

- There is no CPU idle time.
- Because all tasks **execute in parallel**, a multiprogramming system can monitor more quickly.
- Response time is reduced.
- Increases a computer's total job throughput.

❖ **DISADVANTAGES**

- Long-term occupations may need a **lengthy wait**.
- It can be tough to keep track of all processes.
- CPU scheduling is required, as is effective memory management.
- During execution, there is no user interaction with any program.

► Types of Operating System ► **Real Time Operating System** :

- ❖ A real-time operating system (RTOS) is designed to **handle tasks within strict time constraints.**
- ❖ Often used as a control device in a dedicated application such as industrial robots, spacecraft, medical imaging systems, industrial control and scientific research equipment.
- ❖ It **guarantees that critical tasks are completed within a specified time frame**, making it essential for systems where timing is crucial.
- ❖ It has two types:
 - **Soft Real-time OS**
 - A soft real-time operating system aims to meet deadlines most of the time **but allows for occasional delays.**
 - It prioritizes responsiveness and performance but does not guarantee strict adherence to deadlines.
 - **Hard Real-time OS**
 - A hard real-time operating system guarantees that critical tasks are **completed within specified deadlines without any exceptions.**
 - Missing a deadline could lead to **severe consequences**, making strict adherence to timing constraints essential.

► Types of Operating System ► Time Sharing Operating System :

❖ Problem with Multiprocessing OS

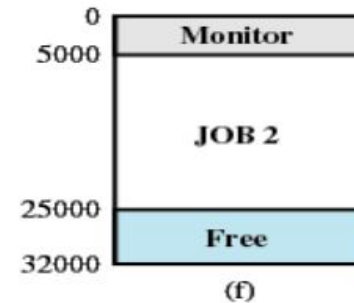
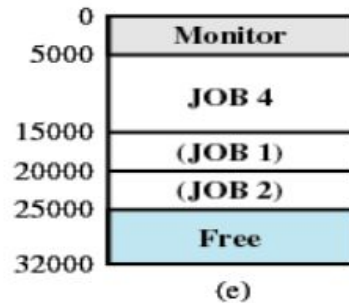
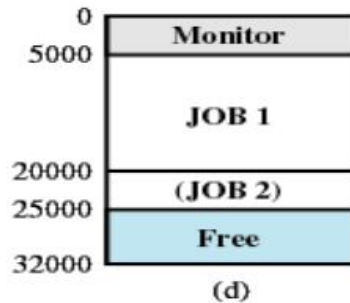
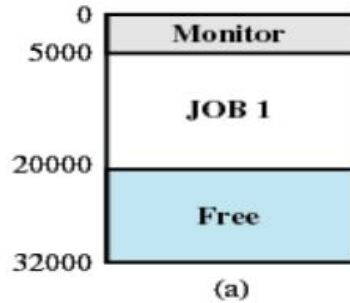
- As more users want to interact with programs while they are running, **it becomes a challenge.**
- Human time is expensive; users don't want to wait.

❖ Solution

- **Timesharing** (a logical extension variant of Multiprogramming)
- Time-sharing is sharing a computing resource among many users by multitasking.
- The CPU executes **multiple jobs by switching among them**, but the switches occur so frequently that the user can interact with each program while it is running.
- The OS provides each user with **one time slice** (slot or share), and each process executes for **one time slot in one round.**
- The CPU switches from one job to another at the end of each time slice.
- The switching is so fast that the **user gets the illusion** that the CPU is executing only one user's job at a time.

► Types of Operating System ► Time Sharing Operating System:

- First time-sharing system developed at MIT (Compatible Time-Sharing System - CTSS)



► Types of Operating System ► Time Sharing Operating System:

❖ Advantage

- Users can easily submit jobs and get immediate feedback.
- Improve user's response time (interaction).

❖ Disadvantages

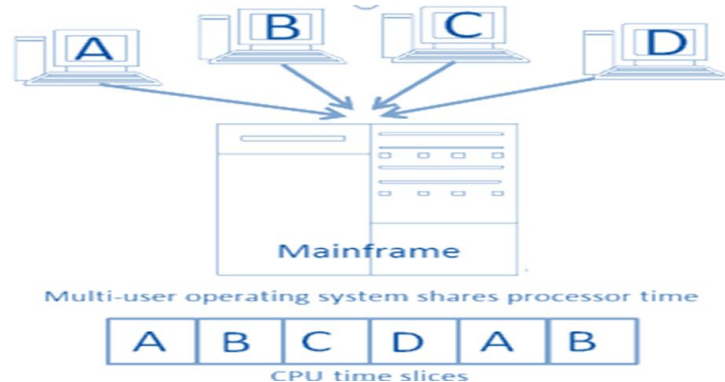
- **Potential for Starvation**(not fair scheduling).
- Overhead of **Context Switching**.
- Complexity of the Operating System.

► Types of Operating System ► **PC Operating System** :

- ❖ It is used in a **dedicated machine** per user where CPU utilization is not a prime concern.
- ❖ The main mode of use of a **PC is by a single user** (as the name personal computer suggests):
- ❖ The system of a PC consists of **two parts**:
 - **BIOS (Basic Input Output System)**: Stored in ROM.
 - **DOS (Disk Operating System)**: Stored on a floppy disk or hard disk.
- ❖ BIOS performs the **power-on self-test (POST)**. After this, it reads a small portion of the OS from the disk, known as the **boot portion**.
- ❖ DOS handles tasks such as **file management, disk management, and CPU management**.
- ❖ There are different types of OSs for PC such as **Windows, MacOS, Linux and Unix**.
- ❖ OS functionality changes with hardware and users.

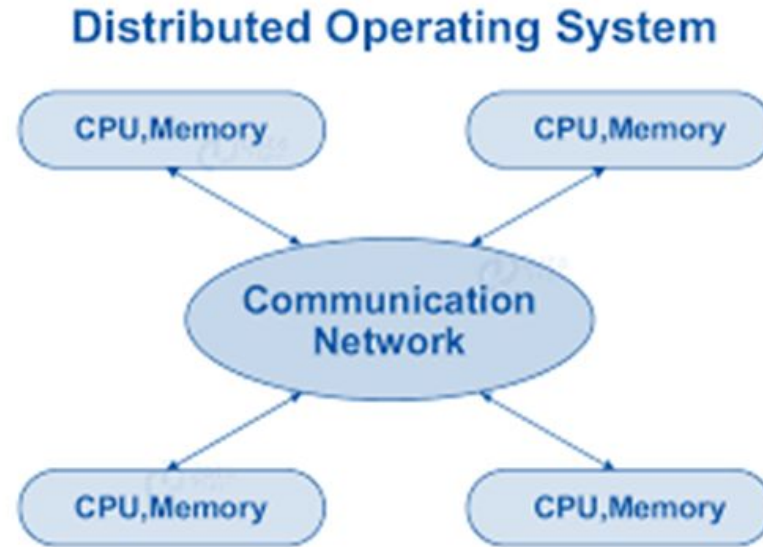
► Types of Operating System ► Mainframe Operating System:

- ❖ Capable of **processing large amounts of information and supporting** a great number of users.
- ❖ Used generally in support of government and business projects.
- ❖ These computers distinguish themselves from PCs in terms of their **I/O capacity**.
- ❖ A mainframe with **1000 disks and thousands of gigabyte of data** is not unusual.
- ❖ They may be used as high-end web servers, servers for large-scale e-commerce sites and servers for business to business transactions.
- ❖ Are heavily oriented towards processing many jobs at once, most of which need prodigious amounts of I/O.
- ❖ They typically offer three kinds of services: **batch, transaction processing and time sharing**.

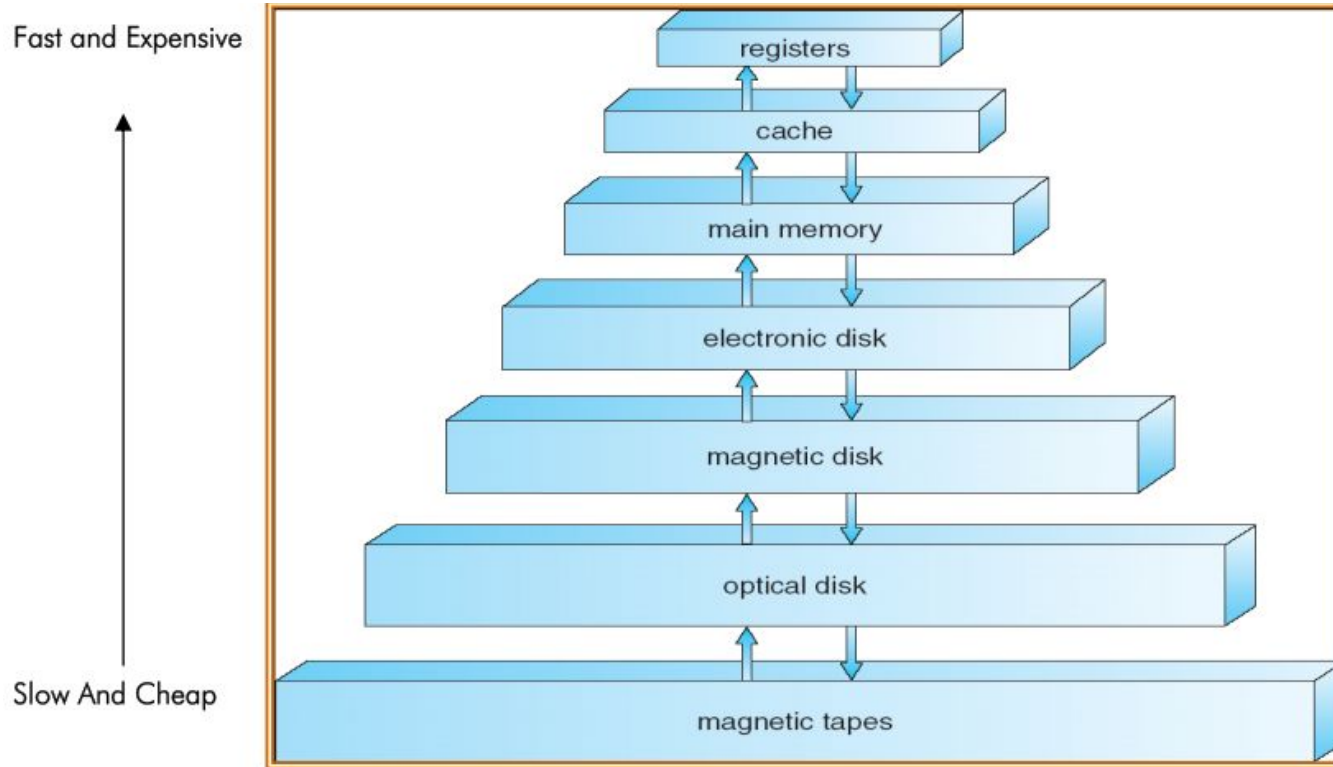


► Types of Operating System ► **Distributed Operating System** :

- ❖ It is the logical extension of a multiprocessor system.
- ❖ In such a system, all computation of jobs is distributed among several processors.
- ❖ Here, **processors do not share memory and clock**. Instead, they communicate with one another via communication lines.



► Memory Hierarchy (Revisit):



► Memory Hierarchy (Revisit):

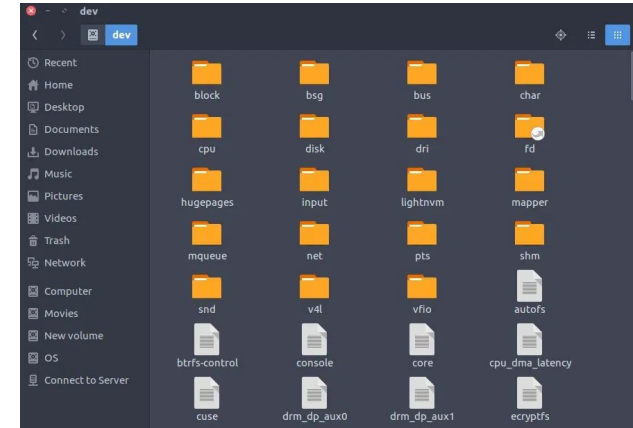
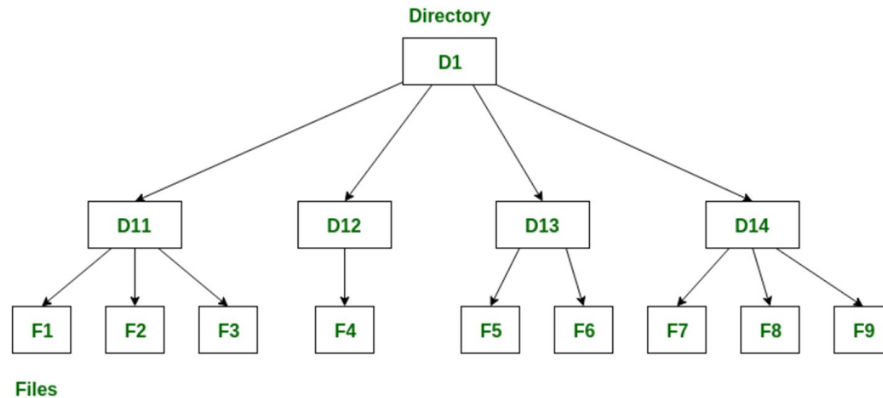
Performance:

Level	1	2	3	4
Name	registers	cache	main memory	disk storage
Typical size	< 1 KB	> 16 MB	> 16 GB	> 100 GB
Implementation technology	custom memory with multiple ports, CMOS	on-chip or off-chip CMOS SRAM	CMOS DRAM	magnetic disk
Access time (ns)	0.25 – 0.5	0.5 – 25	80 – 250	5,000.000
Bandwidth (MB/sec)	20,000 – 100,000	5000 – 10,000	1000 – 5000	20 – 150
Managed by	compiler	hardware	operating system	operating system
Backed by	cache	main memory	disk	CD or tape

► Concepts ► Files :

❖ Files

- The major function of the OS is to hide the internal details of the disks and other I/O devices and present the programmer with a nice , **clean abstract model** of device independent files.
- Every file within the directory hierarchy can be specified by giving it's path name from the top of the directory hierarchy. **For eg: `c:\user\Desktop`**
- **System calls** are obviously needed to create files , remove files , read files and write files.



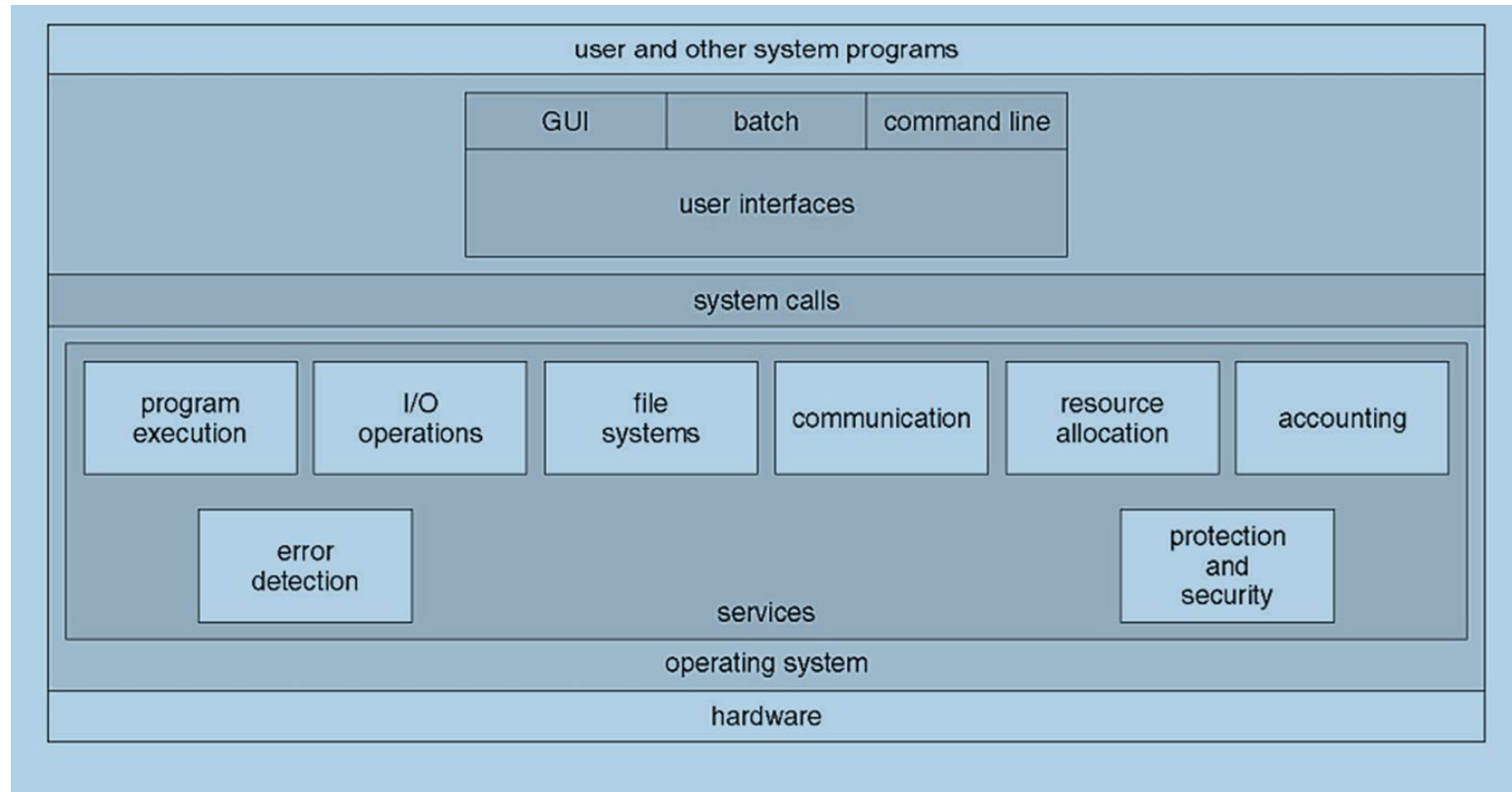
► System Calls:

- ❖ **Problem:** On modern operating systems, **processes do not talk to hardware directly**, but must go through OS.

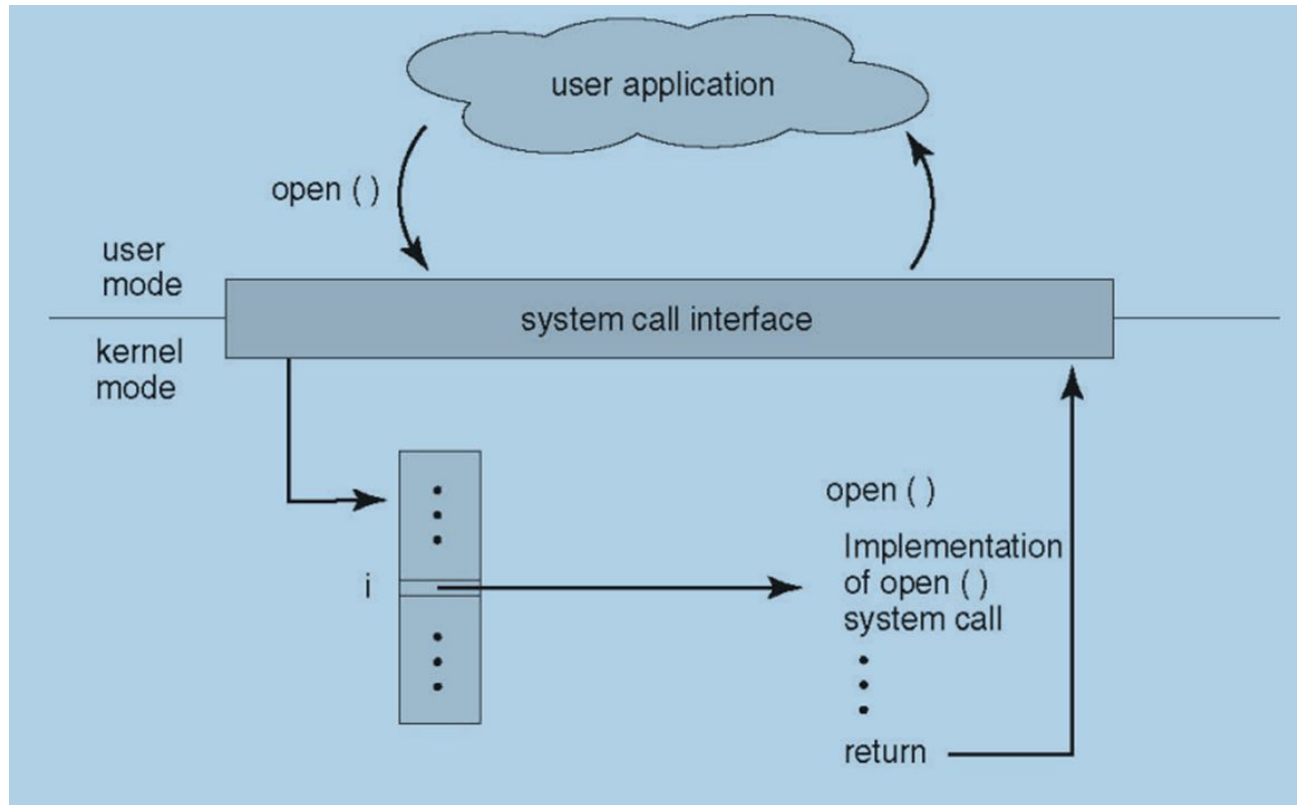
- ❖ **System Calls**

- A system call is how a program requires a **service from an operating system kernel**.
- System call provide the **interface** between a process and the operating system.
- Request from a process for OS to do some kind of work on its behalf.
- The interface between a process and an operating system is provided by system calls.
- In general, system calls are available as **assembly language instructions**.
- To call upon the services of the **operating system** we use system calls.

► System Calls:



► System Call Interface



► Types of System Calls

- ❖ There are mainly five types of system calls.
 - **Process Control:** These system calls deal with processes such as process creation, process termination, load, execute process etc.
 - **File Management:** These system calls are responsible for file manipulation such as creating a file, reading a file, writing into a file, delete file etc.
 - **Device Management:** These system calls are responsible for device manipulation such as reading from device buffers, writing into device buffers, request device, release device.
 - **Information Maintenance:** These system calls handle information and its transfer between the operating system and the user program. **Example:** get/set time or date
 - **Communication:** These system calls are useful for **interprocess communication**. They also deal with creation and deleting a communication connection. **Example:** send and receive message, transfer status information.
 - **Protection:** Protection provides mechanisms for (**access control**) controlling which users/processes have access to which system resources.

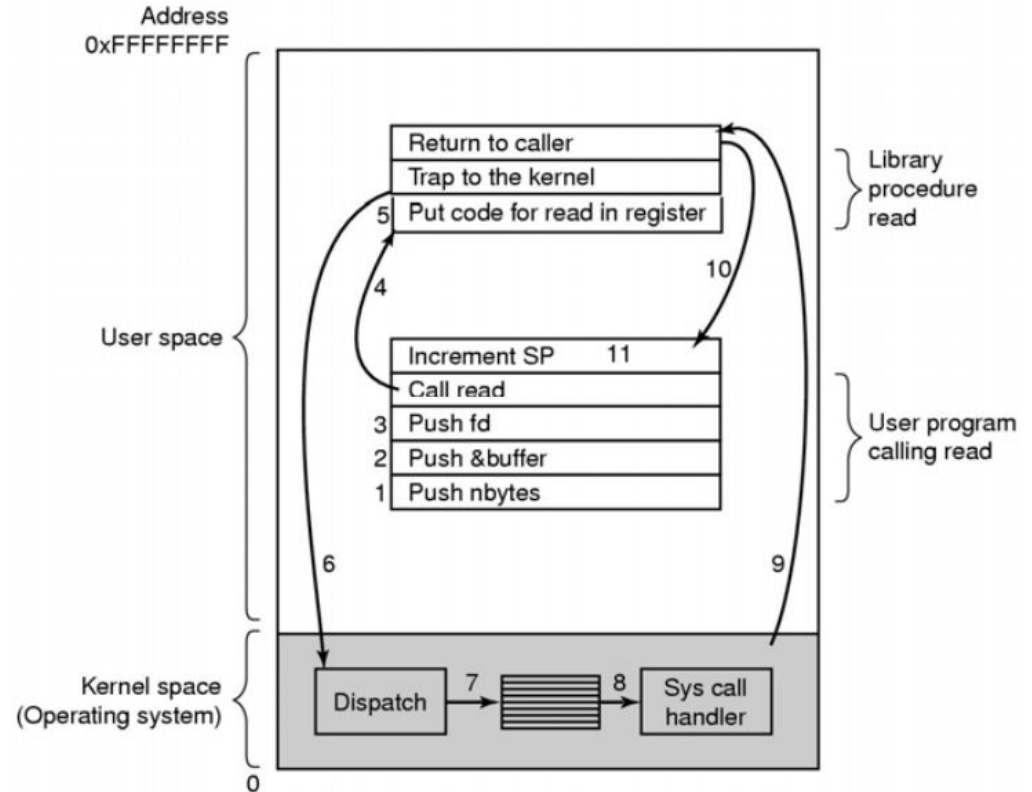
► Types of System Calls

Types of System Calls	Windows	Linux
Process Control	CreateProcess(), ExitProcess(), WaitForSingleObject()	fork(), exit(), wait()
File Management	CreateFile(), ReadFile(), WriteFile(), CloseHandle()	open(), read(), write(), close()
Device Management	SetConsoleMode(), ReadConsole(), WriteConsole()	ioctl(), read(), write()
Information Maintenance	GetCurrentProcessID(), SetTimer(), Sleep()	getpid(), alarm(), sleep()
Communication	CreatePipe(), CreateFileMapping(), MapViewOfFile()	pipe(), shmget(), mmap()

► Steps in Making a System Call:

- ❖ There are **11 steps** in making the system call :

`read (fd, buffer, nbytes)`



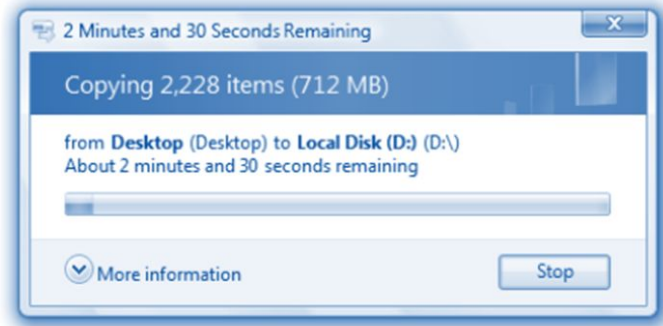
► Steps in Making a System Call (Detailed):

1. **Push nbytes:** The number of bytes to read is pushed onto the stack.
2. **Push &buffer:** The address of the buffer (where data will be stored) is pushed.
3. **Push fd:** The file descriptor (fd) from which to read is pushed onto the stack.
4. **Call read:** The user program calls the read function, which is a library procedure.
5. **Put code for read in register:** The library procedure places the system call number for read in a specific CPU register.
6. **Trap to the kernel:** A special CPU instruction is executed to **switch from user mode to kernel mode**, transferring control to the kernel.
7. **Dispatch:** The kernel uses the system call number (from the register) to locate the appropriate system call handler in a system call table.
8. **System Call Handler:** The specific read system call handler in the kernel executes. It reads from the file, accesses the buffer address passed, and fills the buffer with data.
9. **Return from kernel:** Control returns from the kernel back to user space once the system call is completed.
10. **Return to caller:** The library procedure returns control back to the original calling user program.
11. **Increment SP:** The user program increments the **stack pointer** to clean up arguments pushed earlier (like fd, &buffer, nbytes).

► System Call (Example)

Example-2: Sequence of system calls to copy contents of one file to another.

1. Display dialog to choose source file
2. Display dialog to choose destination folder
3. Display progress dialog
4. Open source file for reading
5. Open destination file for creating and writing
6. Loop while more source bytes remaining:
 1. Read n bytes from source file
 2. Write n bytes to destination file
 3. Update progress dialog
7. Close source file
8. Close destination file
9. Close progress dialog
10. Terminate normally



► Operating System Structure

- ❖ The operating system structure is a way in which the **various components of an operating system are organized and interconnected.**
 - OS structure depends on component organization and interaction.
 - Acts as a blueprint that guides OS design.
 - Breaking the OS into smaller parts simplifies development.
 - Structure defines how components are integrated into the kernel.
- ❖ The structure of an operating system can be classify as the **following structures:**
 - Simple Structure
 - Monolithic Structure
 - Microkernel Structure
 - Exo-Kernel Structure
 - Layered Structure
 - Virtual Machines

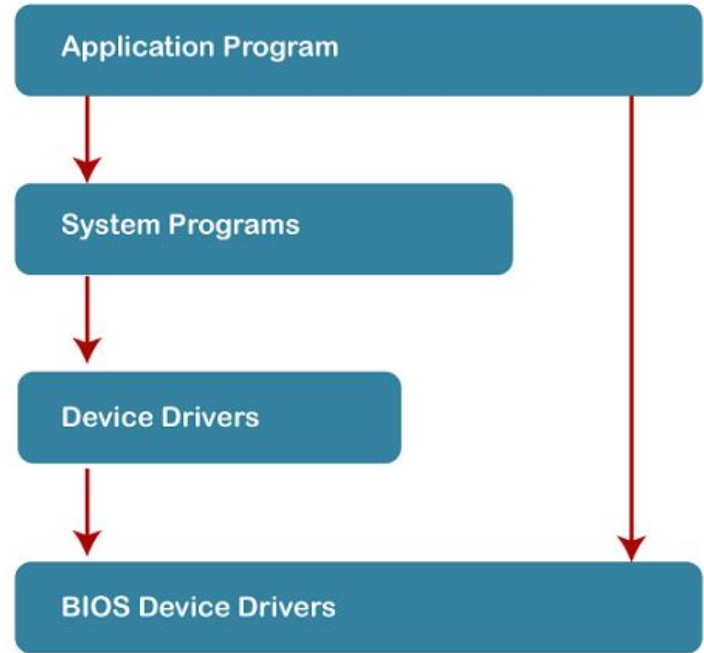
► Operating System Structure > Simple Structure :

- ❖ It is the most straightforward operating system structure, but have no well-defined architecture and are small, simple and limited in functionality.
- ❖ This organizational structure is used by the MS-DOS operating system.
- ❖ Their interfaces and functional levels are not clearly separated, which makes them less reliable.
- ❖ Because of this lack of separation, a failure in any user program can cause the entire system to crash.
- ❖ Because MS-DOS systems have a low level of abstraction, programs and I/O procedures are visible to end users, giving them the potential for unwanted access.
- ❖ **Example:** MS-DOS Operating System

► Operating System Structure ► Simple Structure :

- ❖ There are four layers that make up the MS-DOS operating system, and each has its own set of features. These layers are:

1. Application Program
2. System Program
3. MS-DOS device drivers
4. ROM BIOS device drivers



► Operating System Structure ► Monolithic Structure :

- ❖ A monolithic structure is an OS architecture in which the **entire system is built as one large kernel.**
- ❖ All major services such as process management, memory management, file systems and device drivers run together in a **single code base.**
- ❖ This design places all core functionality inside **one unified kernel.**
- ❖ The components of monolithic operating system are **organized haphazardly** and any module can call any other module without any reservation.
- ❖ **The structure is that there is no structure.**
- ❖ The operating system is written as a collection of procedures, each of which can call any of the other ones whenever it needs to.
- ❖ **Examples:** Linux, Unix, Windows NT, Windows 95

► Operating System Structure ► **Monolithic Structure** :

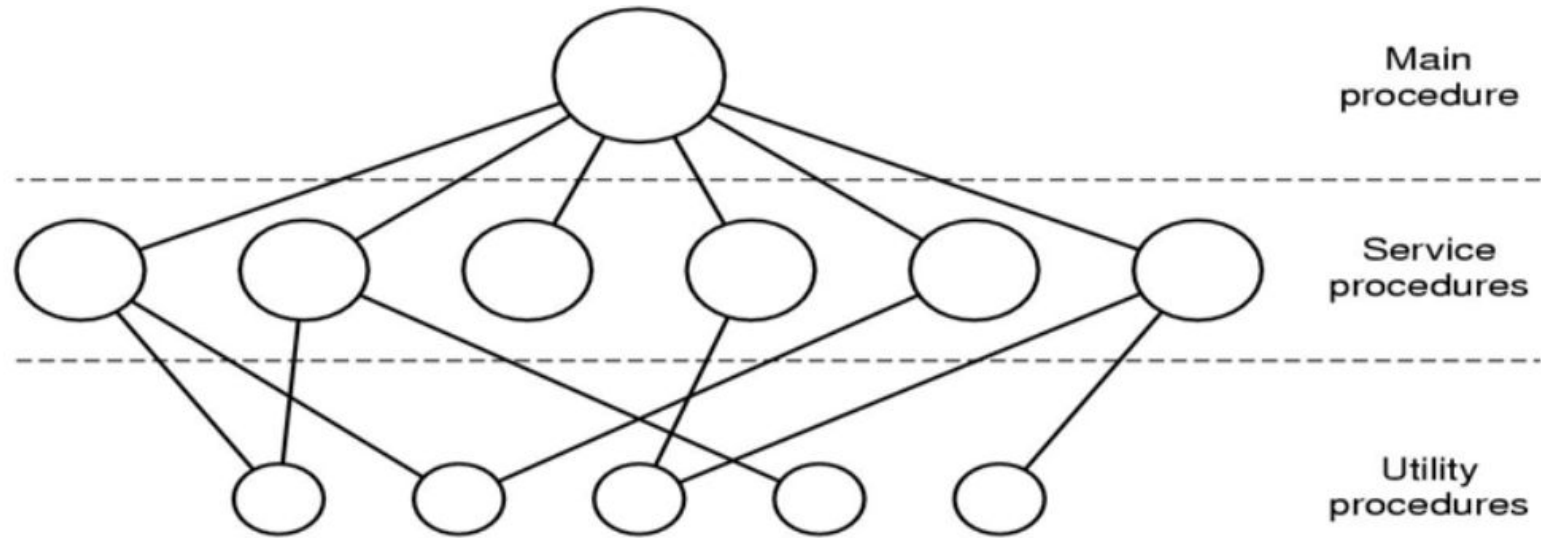


Fig: A simple structuring model for a monolithic system

► Operating System Structure ► **Monolithic Structure :**

◆ Advantages of Monolithic Structure

- It is simple to design and implement because all operations are managed by kernel only, and layering is not needed.
- As services such as memory management, file management, process scheduling, etc., are implemented in the **same address space**, the execution of the monolithic kernel is **relatively fast** as compared to normal systems.
- Simple design and implementation.

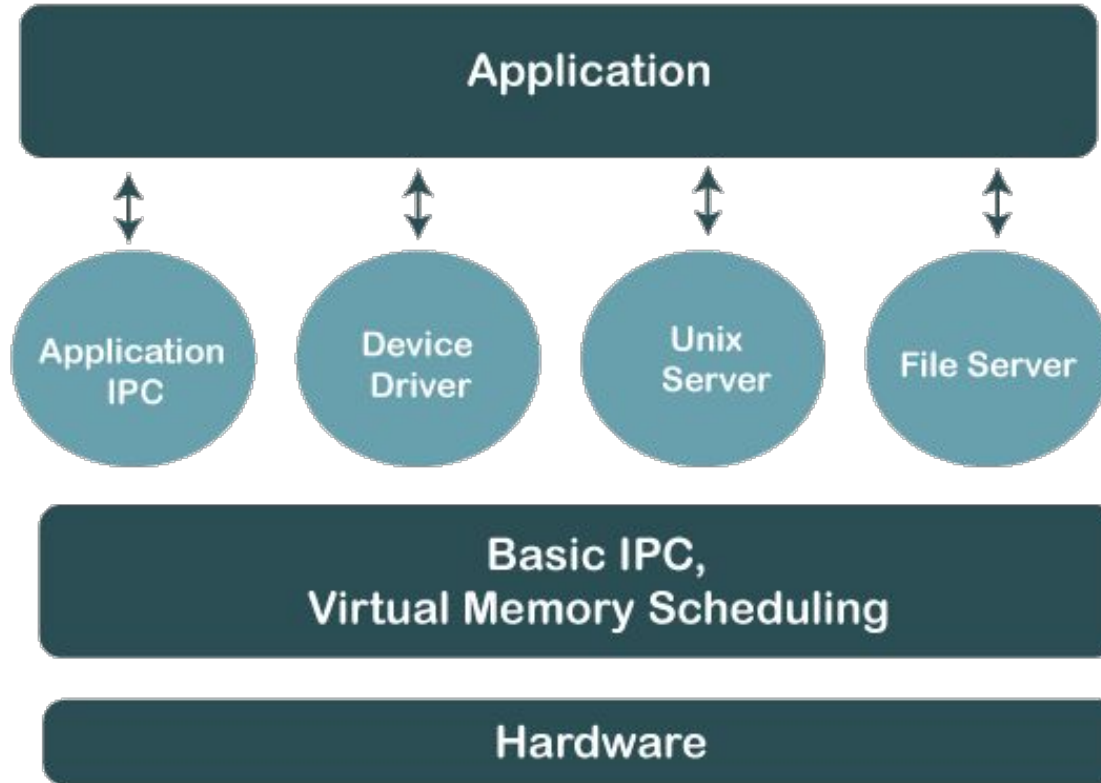
◆ Disadvantages of Monolithic Structure

- Fault in any single component has the potential to **bring down the whole system** since there is no isolation between different parts.
- This can be a hard and unsafe task, as **changes in one part** of the system may accidentally filter out into another part.
- Since the **components are more interdependent**, debugging is also more complex and always requires heavy testing and testing-related activities.

► Operating System Structure ► **Microkernel Structure :**

- ❖ A Microkernel is a way of designing an Operating System where only the **most important parts** are kept inside the kernel.
- ❖ A Micro-Kernel structure removes all non-essential services from the kernel and runs them in user space, **leaving only a small core** called the **microkernel**.
- ❖ All other parts, like device drivers and file systems, are moved outside the kernel and **run as separate** programs.
- ❖ If one part (Microkernel service) stops working or crashes, the rest of the **operating system keeps running** normally without problems.
- ❖ Features:
 - Non-essential services run in user space, leaving a small core kernel.
 - New services can be added without changing the kernel.
 - More secure and reliable because failures don't crash the whole OS.
- ❖ **Example:** macOS and QNX

► Operating System Structure > **Microkernel Structure** :



► Operating System Structure ► **Microkernel Structure :**

❖ Advantages of Micro-kernel structure

- The just necessary services running in the kernel ensure that user service failures don't crash the whole system.
- It isolates the services from the kernel, thus minimizing the possibility of compromising the system since every service operates independently of the other.
- Smaller kernels are easier to operate and alter. Hence, updating and bug fixes are less challenging.
- Services can be added or removed without affecting the kernel. It gives more flexibility while designing the system.

❖ Disadvantages of Micro-kernel structure

- The inter-process communication between kernel and user services introduces additional overhead and hence is slower as compared to the monolithic approach.
- Separation of services must be done very carefully, which may complicate the initial development process.
- Context switches are more frequent between user space and kernel space in this approach; hence, performance would be slower in highly loaded conditions.

► Monolithic VS Microkernel:

Monolithic kernel

It is a large process running in a single address space

In order to write a monolithic kernel, less code is required.

If a service crashes, the whole system collapses in a monolithic kernel.

It is a single static binary file

Linux, BSDs, Microsoft Windows (95,98, Me), Solaris, OS-9, AIX, DOS, XTS-400, etc.

MicroKernel

It can be broken down into separate processes called servers.

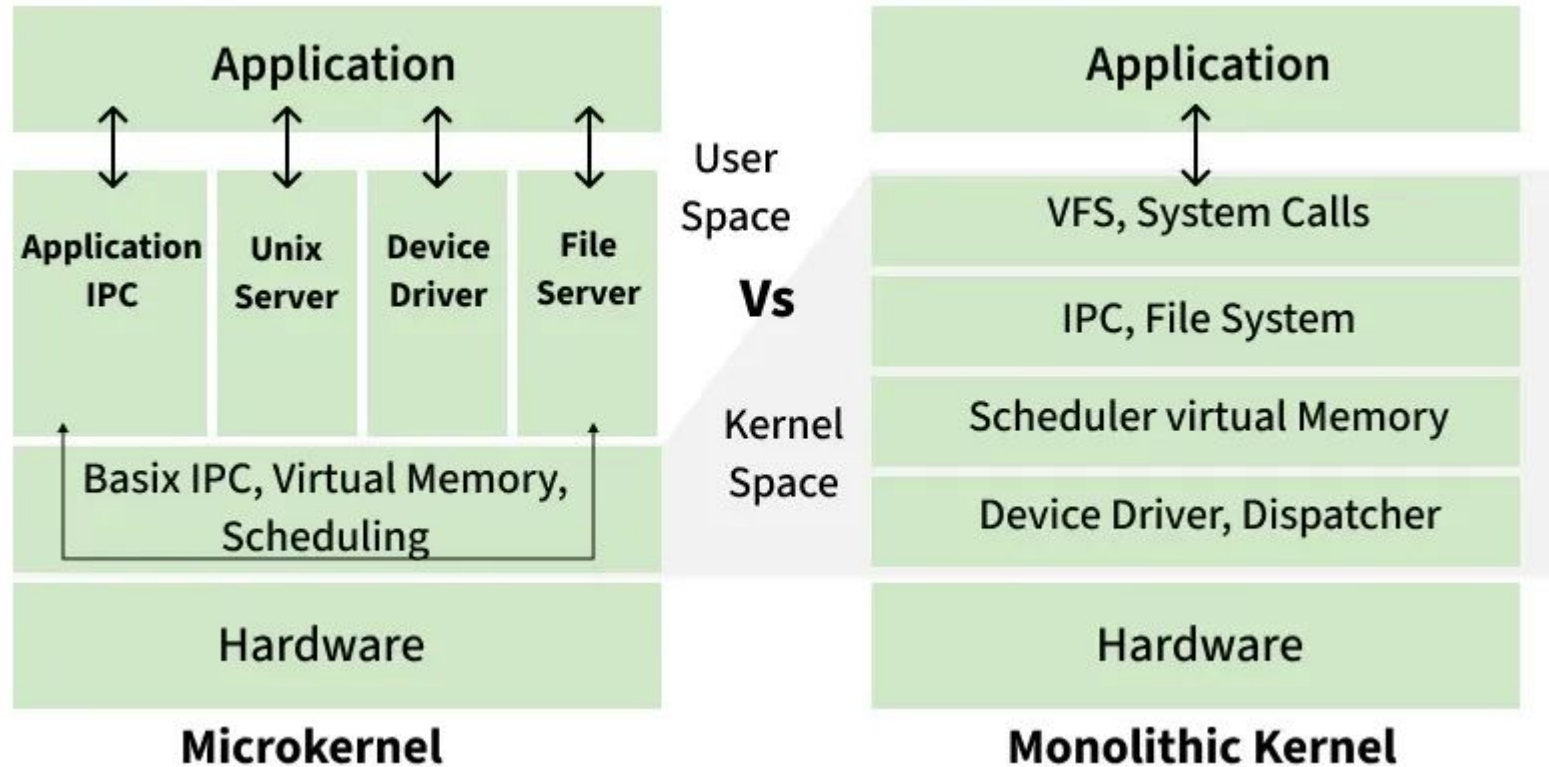
In order to write a microkernel, more code is required

If a service crashes, it never affects the working of a microkernel.

Servers communicate through IPC.

L4Linux, QNX, SymbianK42, Mac OS X, Integrity, etc.

► Monolithic VS Microkernel:

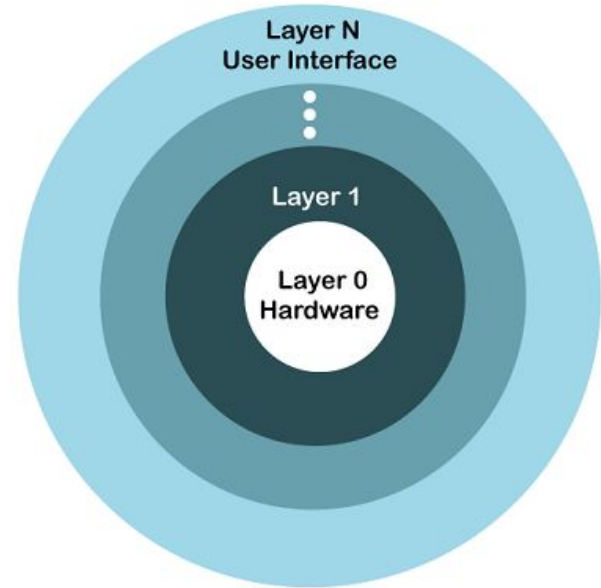


► Operating System Structure > Layered Structure :

- ❖ In a layered operating system structure, the **OS is divided into hierarchical layers**, each with specific responsibilities.
- ❖ The hardware resides at the bottom layer (Layer 0), while the user interface is at the top layer (Layer N).
- ❖ The Layered Structure of the Operating System is a way of organizing the different components of an OS into separate layers.
- ❖ Each layer in between serves specific functions and communicates only with the **layers directly above and below it**.
- ❖ This hierarchical design ensures that each layer can be **developed and debugged independently**, with changes in one layer typically not affecting others.

► Operating System Structure ► **Layered Structure :**

Layer	Function
5	The operator
4	User programs
3	Input/output management
2	Operator-process communication
1	Memory and drum management
0	Processor allocation and multiprogramming



► Operating System Structure ► **Layered Structure :**

❖ Advantages of Layered Structure:

- Development, maintenance, and understanding are much easier, as every layer is assigned a specific role.
- It can isolate problems in individual layers because testing and debugging would be less demanding.
- Modifying or updating one layer will not affect other independent layers directly; thus, upgrading or fixing could be done more easily.

❖ Disadvantages of Layered Structure:

- The layered approach adds delays due to the fact that data and commands must pass through several layers, thus slowing down the system in general.
- Designing a properly working layered structure requires forethought and iron discipline for proper interaction between different layers.
- Layers are generally dependent on lower layers; thus skipping layers and directly accessing the lower-level functionalities becomes harder, hence reducing flexibility in many systems.

► Operating System Structure ► Virtual Machines :

- ❖ A virtual machine (VM) is a **software-based emulation** of a physical computer.
- ❖ It runs an operating system and applications just like a physical computer, but it is **isolated from** the underlying physical hardware by a **layer of software** called a *hypervisor* or *Virtual Machine Monitor (VMM)*.
- ❖ Layers:
 1. Hardware
 - At the bottom layer, this represents the physical hardware of the system.
 - It includes the physical CPU, memory, storage, network interfaces, and other physical resources.
 2. Host OS/VMM
 - This layer can be either a Host Operating System (OS) or a Virtual Machine Monitor (VMM), also known as a hypervisor.
 - The role of this layer is to manage the hardware resources and provide an environment where virtual machines (VMs) can operate.

► Operating System Structure ► Virtual Machines :

There are two types of hypervisors

- **Type 1 Hypervisor** : Runs directly on the hardware and does not require a host operating system. Examples: VMware ESXi, Microsoft Hyper-V, and Xen.
- **Type 2 Hypervisor** : Runs on top of a host operating system. Examples : VMware Workstation and Oracle VirtualBox.

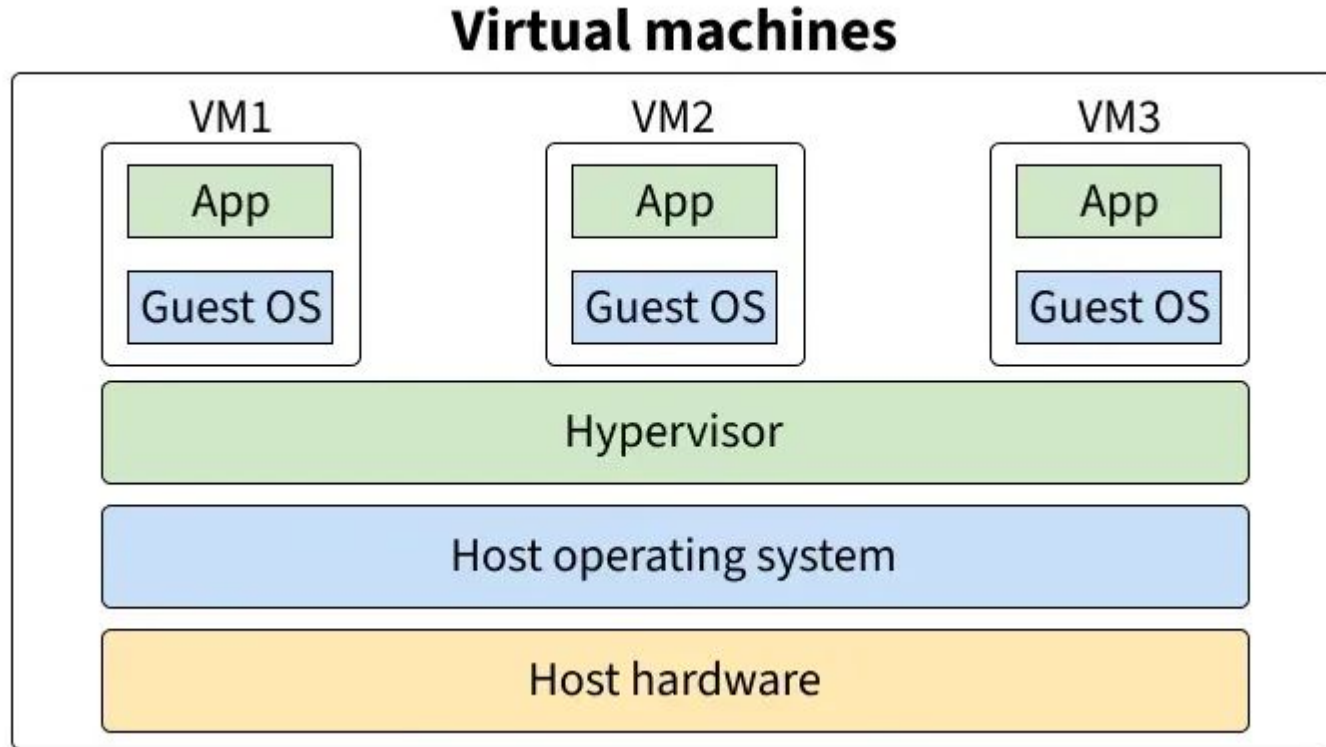
4. VMs (Virtual Machines)

- These are the virtual instances created by the VMM or hypervisor.
- Each VM is an independent entity and can run its own guest operating system.
- In the diagram: VM1, VM2, ..., VMi, VMn, these represent individual virtual machines.
- The number of VMs (n) can vary depending on the resources and the capabilities of the hypervisor.

5. Guest OS

- Each virtual machine has its own Guest OS, which is the operating system running inside the VM.
- The Guest OS can be the same or different from the Host OS. It interacts with the virtual hardware provided by the VMM and can run applications as if it were running on a physical machine.

► Operating System Structure ► **Virtual Machines :**



► Operating System Structure ► **MCQs:**

In which OS structure do most services run in user space rather than kernel space?

- ☐ (A) Monolithic
- ☐ (B) Microkernel
- ☐ (C) Hybrid
- ☐ (D) Simple

► Operating System Structure ► MCQs:

Which OS structure organizes components into layers where each layer uses services of the lower layer?

- ☐ A Modular
- ☐ B Layered
- ☐ C Exokernel
- ☐ D Virtual Machine

► Kernel and it's Types:

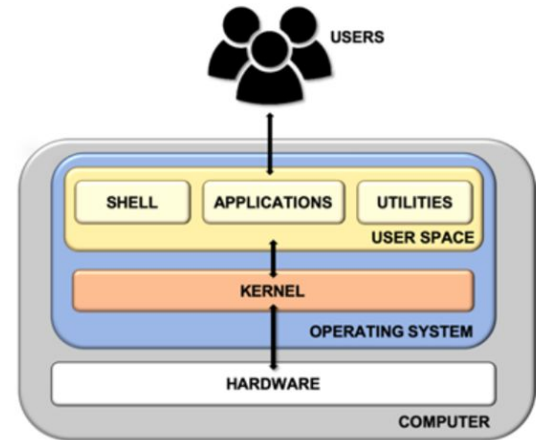
- ❖ Kernel is a **core component** of Operating System **without it OS can't work.**
- ❖ Kernel is the nervous system of OS.
 - It is the central core of Operating System.
 - It controls everything in OS including *I/O Management, Process Management* and *so on.*
 - It is a **bridge** between *applications and the actual data processing* done at the hardware level.
 - It acts as an interface between the user applications and the hardware.
- ❖ The types of Kernel are listed below:
 1. Monolithic Kernel
 2. Micro Kernel
 3. Hybrid Kernel
 4. Exo- kernel
 5. Nano Kernel

► Kernel ► Functions :

- ❖ The major functions of Kernel are:
 - The kernel provides a mechanism for the creation and deletion of processes.
 - It provides *CPU scheduling, memory management, and device management* for these processes.
 - The kernel provides synchronization tools so that processes can synchronize their actions.
 - It provides communication tools so that *processes can communicate* with each other.

► OS Concepts ► Shell:

- ❖ A shell in an operating system (OS) is a **user interface** that provides access to various services of the OS.
- ❖ It serves as an intermediary between the **user and the OS kernel**, allowing users to *interact with the system, run commands, manage files, and execute programs*.
- ❖ Generally two types of Shell
 - GUI(Graphical User Interface)
 - CLI(Command Line Interface)



► Shell ► **GUI** (Graphical User Interface):

- ❖ A graphical shell provides a graphical user interface (GUI), allowing users to interact with the OS through **graphical elements like windows, icons, and menus**.
- ❖ This type of shell is **more user-friendly**, particularly for those less familiar with command-line interfaces.
- ❖ **Examples of GUI**
 - **Windows Explorer**
 - The graphical shell for Microsoft Windows.
 - **GNOME, KDE**
 - Popular graphical shells for Linux systems, providing desktop environments.

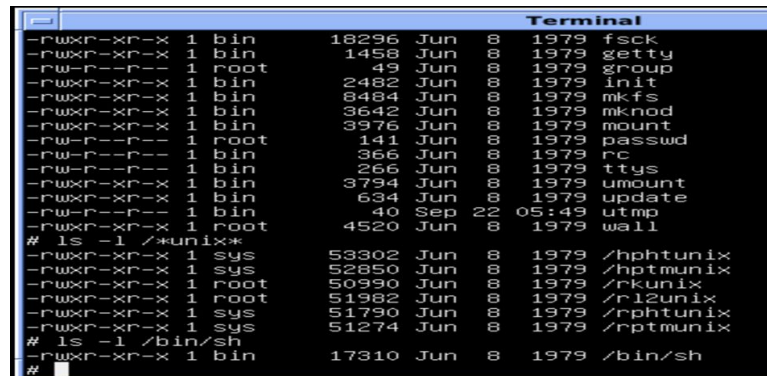
► Shell ► **CLI** (Command Line Interface):

- ❖ A command-line shell provides a **text-based interface** where users can type commands to perform specific tasks.
- ❖ It is a powerful tool for interacting with the operating system, **offering greater control** over system functions.
- ❖ **Examples of command-line shells are**
 - **Bash (Bourne Again Shell):** Commonly used on **Unix-like systems**, including Linux and macOS.
 - **Cmd.exe:** The **default command-line interpreter** for Windows.
 - **PowerShell:** A **more advanced command-line shell** developed by Microsoft for Windows, providing more powerful scripting capabilities.
 - **Zsh (Z Shell):** Known for its advanced features, such as improved **auto-completion** and scripting capabilities. It is the default shell on **macOS**.

► Shell ► **CLI** (Command Line Interface):

❖ Bourne Shell

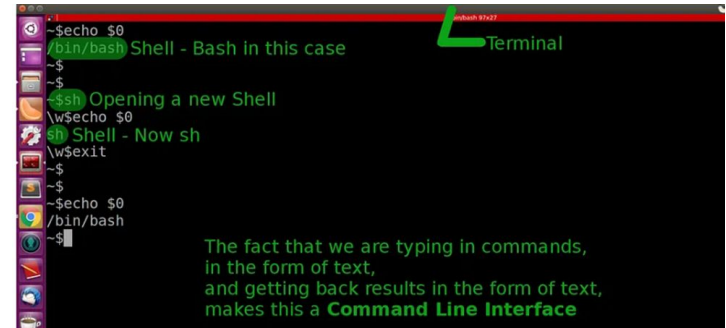
- Used by UNIX operating system.
- It provides command based programming to interpret and execute user commands.



```
Terminal
-rwxr-xr-x 1 bin 18296 Jun 8 1979 fsck
-rwxr-xr-x 1 bin 1458 Jun 8 1979 getty
-rw-r--r-- 1 root 49 Jun 8 1979 group
-rwxr-xr-x 1 bin 2482 Jun 8 1979 init
-rwxr-xr-x 1 bin 8484 Jun 8 1979 mkfs
-rwxr-xr-x 1 bin 3642 Jun 8 1979 mknod
-rwxr-xr-x 1 bin 3976 Jun 8 1979 mount
-rw-r--r-- 1 root 141 Jun 8 1979 passwd
-rw-r--r-- 1 bin 366 Jun 8 1979 rc
-rw-r--r-- 1 bin 266 Jun 8 1979 ttys
-rwxr-xr-x 1 bin 3794 Jun 8 1979 umount
-rwxr-xr-x 1 bin 634 Jun 8 1979 update
-rw-r--r-- 1 bin 40 Sep 22 05:49 utmp
-rwxr-xr-x 1 root 4520 Jun 8 1979 wall
# ls -l /*unix*
-rwxr-xr-x 1 sys 53302 Jun 8 1979 /hptunix
-rwxr-xr-x 1 sys 52850 Jun 8 1979 /hptmunix
-rwxr-xr-x 1 root 50990 Jun 8 1979 /rkunix
-rwxr-xr-x 1 root 51982 Jun 8 1979 /rl2unix
-rwxr-xr-x 1 sys 51790 Jun 8 1979 /rphtunix
-rwxr-xr-x 1 sys 51274 Jun 8 1979 /rptmunix
# ls -l /bin/sh
-rwxr-xr-x 1 bin 17310 Jun 8 1979 /bin/sh
#
```

❖ Bash Shell (Bourne Again Shell).

- Bash shell is the free and enhanced version of the Bourne shell.
- Command language interpreter for the GNU
- It currently runs on nearly every version of UNIX and a few other OS. (MS-DOS , Windows platform etc).



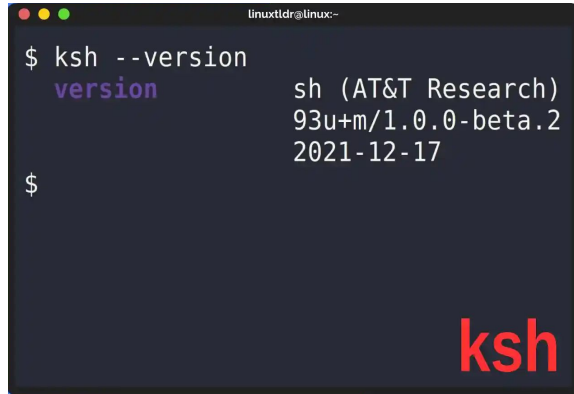
```
Terminal
$echo $0
/bin/bash
$Shell - Bash in this case
$
$ssh Opening a new Shell
$echo $0
$Shell - Now sh
$exit
$
$
$echo $0
/bin/bash
$

The fact that we are typing in commands,
in the form of text,
and getting back results in the form of text,
makes this a Command Line Interface
```


► Shell ► **CLI** (Command Line Interface):

❖ Korn Shell

- Korn Shell or KSH which attempts to integrate the features of other shells like C shell, Bourne Shell, etc.
- It provides more programming features than Bash.

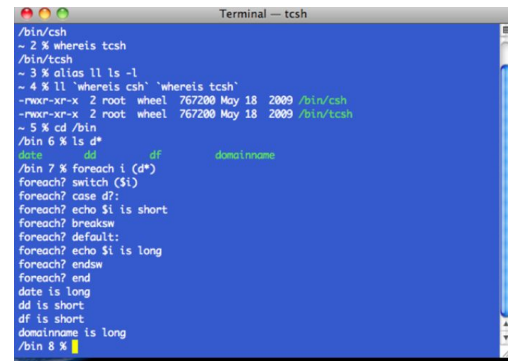


```
linuxtidr@linux:~$ ksh --version
version          sh (AT&T Research)
                  93u+m/1.0.0-beta.2
                  2021-12-17
$
```

ksh

❖ C shell

- The C shell is a command processor which is typically run in a text window, allowing the user to type and execute commands.
- The overall style of the language looked more like C and was seen as more readable.
- Example: macOS and Red Hat Linux csh is used .



```
Terminal - tcsh
/bin/csh
~ 2 % whereis tcsh
/bin/tcsh
~ 3 % alias ll ls -l
~ 4 % ll `whereis csh` `whereis tcsh`
-rwxr-xr-x  2 root  wheel  767200 May 18  2009 /bin/csh
-rwxr-xr-x  2 root  wheel  767200 May 18  2009 /bin/tcsh
~ 5 % cd /bin
/bin 6 % ls d*
date      dd      df      domainname
/bin 7 % foreach i (d*)
foreach? switch ($i)
foreach? case d?:
foreach? echo $i is short
foreach? breaksw
foreach? default:
foreach? echo $i is long
foreach? endsw
foreach? end
date is long
dd is short
df is short
domainname is long
/bin 8 %
```

► Open Source Operating System:

- ❖ An open-source operating system **allows users to view, modify, and distribute** its source code freely.
- ❖ An operating system whose source code is **not accessible** by the public is known as a closed source operating system
- ❖ **Examples:** *Linux, FreeBSD, Android (AOSP), ReactOS.*
- ❖ **Counter** to the copy protection and Digital Rights Management (DRM) movement.
- ❖ Started by **Free Software Foundation (FSF)**, which has “copyleft” **GNU Public License (GPL)**
- ❖ Examples include GNU/Linux and BSD UNIX (including core of Mac OS X), and many more
- ❖ Can use VMM like VMware Player (Free on Windows), Virtualbox (open source and free on many platforms - <http://www.virtualbox.com>) .

► Open Source Operating System ► **Types :**

1. General-Purpose OS

- For everyday computing, servers, and desktops.
- **Examples:** Ubuntu, Fedora, FreeBSD

2. Specialized OS

- Built for specific tasks (security, networking).
- Examples:
 - Qubes OS (security-focused).
 - PfSense (firewall/router).
 - Real-Time OS (RTOS)
 - Predictable timing for robotics/automotive.
 - Example: FreeRTOS.

► Operating System ► Functions :

❖ Following are some of the important functions of an Operating System:

1. Memory Management
2. Process Management
3. Device Management
4. File Management

Major Functions

5. Security
6. Control over system performance
7. Job accounting
8. Error detecting aids
9. Coordination between other software and users

Minor Functions

► Memory Management:

- ❖ Memory management refers to the **management** of *Primary Memory* or *Main Memory*.
- ❖ Main memory is a **large array of words or bytes** where each word or byte has its own address.
- ❖ Main memory provides **fast storage** that can be accessed directly by the CPU. So for a **program to be executed**, it **must be in the main memory**.
- ❖ Operating System does the following activities for memory management.
 - Keeps **tracks** of primary memory i.e. what part of it are in use by whom, what part are not in use.
 - In multiprogramming, OS **decides** which process will get memory when and how much.
 - **Allocates** the memory when the process requests it to do so.
 - **De-allocates** the memory when the process **no longer needs** it or has been terminated

► Process Management:

- ❖ In multiprogramming environment, **OS decides** which process gets the processor when and how much time.
- ❖ This function is called **process scheduling**.
- ❖ Operating System does the following activities for processor management.
 - Keeps tracks of processor and status of process.
 - Program responsible for this task is known as **traffic controller**.
 - **Allocates** the processor(CPU) to a process.
 - **De-allocates** processor when processor is no longer required.

► Device Management:

- ❖ In Device Management, OS manages **device communication** via their respective drivers.
- ❖ Operating System does the following activities for device management.
 - Keeps tracks of **all devices**.
 - Program **responsible** for this task is known as the I/O controller.
 - Decides which process gets the device when and for how much time.
 - **Allocates** the device in the efficient way.
 - **De-allocates** devices.

► File Management:

- ❖ A file system is normally organized into directories for easy navigation and usage.
- ❖ These directories may contain files and other directions. Operating System does the following activities for file management.
 - Keeps track of information, location, uses, status etc.
 - The collective facilities are often known as **file system**.
 - **Decides** who gets the resources.
 - **Allocates** the resources.
 - **De-allocates** the resources.

► Remaining Functions:

❖ Following are some of the important activities that Operating System does.

➤ Security

- By means of password and similar other techniques, preventing unauthorized access to programs and data.

➤ Control over system performance

- **Recording delays** between request for a service and response from the system.

➤ Job accounting

- **Keeping track** of time and resources used by various jobs and users.

➤ Error detecting aids

- Production of dumps, traces, error messages and other debugging and error detecting aids.

➤ Coordination between other software and users

- Coordination and assignment of **compilers, interpreters, assemblers** and other software to the various users of the computer systems.



► PYQs:

1. What is Operating System? Discuss Operating System as a resource manager. **2025, 2022 PU (8 Marks)**
2. Explain different types of Operating Systems in detail. **2025, 2023 PU (8 Marks)**
3. What is Operating System? Briefly explain the function and types of Operating System. **2024 PU (8 Marks)**
4. How operating system can be explained as resource manager and extended machine? **2024 PU (8 Marks)**
5. Short Notes:
 - a. History of OS **2024, 2022 PU (4 Marks)**



References:

1. Andrew S. Tanenbaum, "*Modern Operating System*", PHI
2. Silberschatz and Galvin, "*Operating System Concepts*", Addison Wesley
3. Andrew S. Tanenbaum, "*Operating System, Design & Implementation*", PHI

▶ **THANKS!**

? **Do you have any questions?**

✉ **shishir.ghimire@newsummit.edu.np**